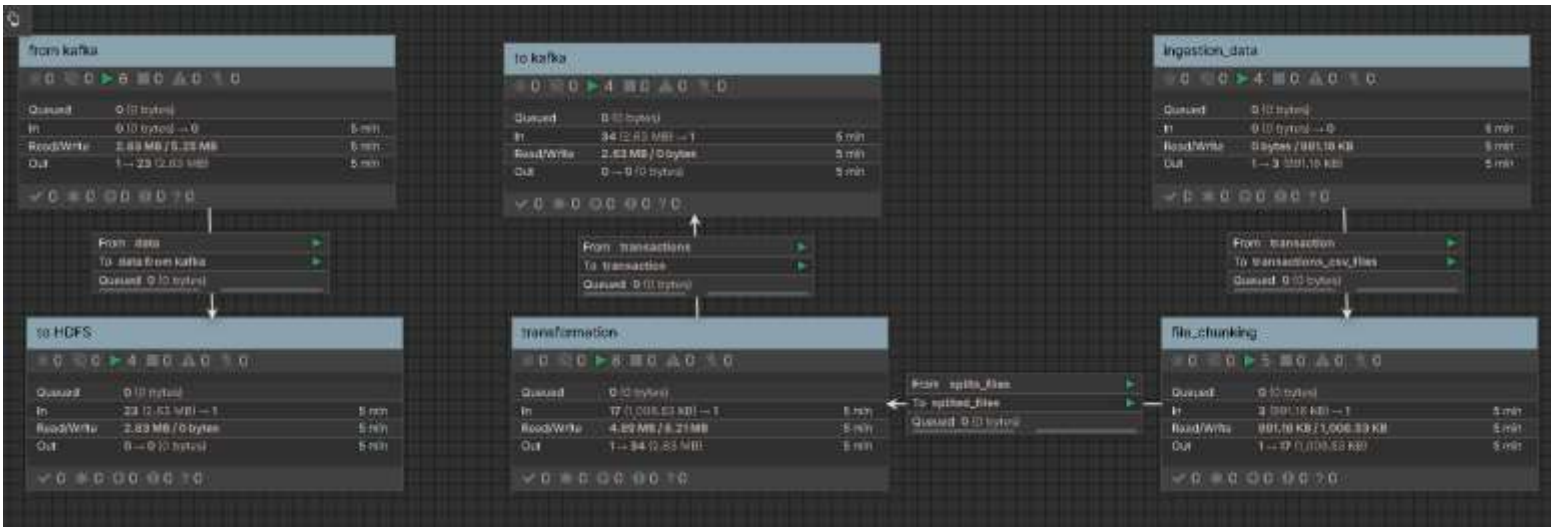
Odai Aqlan

Eng.Ahmed Data Engineering Assignment

Real-Time Data Pipeline Using Apache NiFi, Kafka, and Hadoop

Architecture Diagram

I divide the project into those six process groups and connect them with input and output ports to keep the flow organized and scalable.



ingestion_data :to read CSV files from the /lab_data/input directory
file_chunking : to split large CSV files into manageable chunks (1544 records per split)
transformation :to convert CSV chunks to JSON with validation and null handling
to kafka :to publish cleaned JSON records to Kafka topic: banking-transactions
from kafka **:**to consume messages from Kafka and merge them into batches
to HDFS :to store final JSON files in HDFS partitioned by date

Ask me Why? :)

Because structuring the flow this way, I make it easier to maintain, extend, and scale later and it is common best practice.

I will explain each process group and the decisions down there...

Real-Time Data Simulation

I created a Python script that generates messy, duplicate banking transaction CSV files every 0.001 seconds to simulate real-world transactions. The files get automatically saved into the /lab_data/input directory so NiFi can pick them up.

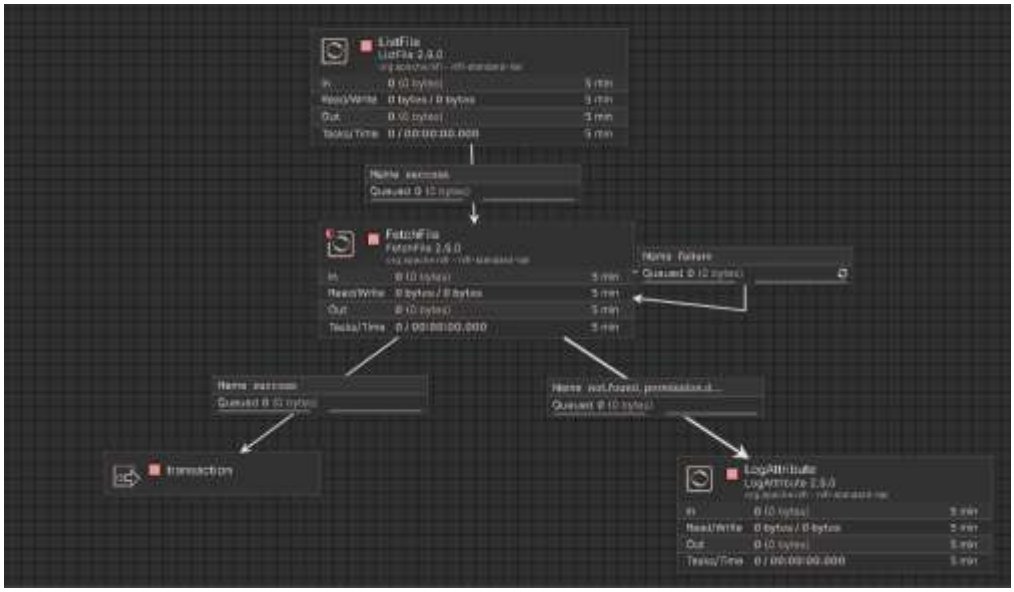
The script is in the same folder out there.

Account_id	Transaction_type	Amount	Currency	Timestamp
ACC050	INVALID	100	SAR	2026-05-16T01:30:05Z
ACC939	DEPOSIT	500	SAR	2026-05-16T01:30:05Z
ACC008	WITHDRAWAL	250		2026-05-16T01:30:05Z
ACC846	TRANSFER	-200	YER	2026-05-16T01:30:05Z
ACC271	TRANSFER	250		2026-05-16T01:30:05Z
ACC307	TRANSFER		YER	2026-05-16T01:30:05Z
ACC307	TRANSFER		YER	2026-05-16T01:30:05Z
ACC307	DEPOSIT	250	USD	2026-05-16T01:30:05Z

NIFI Pipeline

Process Group: ingestion_data

This is the first process group its job is simple: watch a folder and fetch new CSV files the moment they appear.



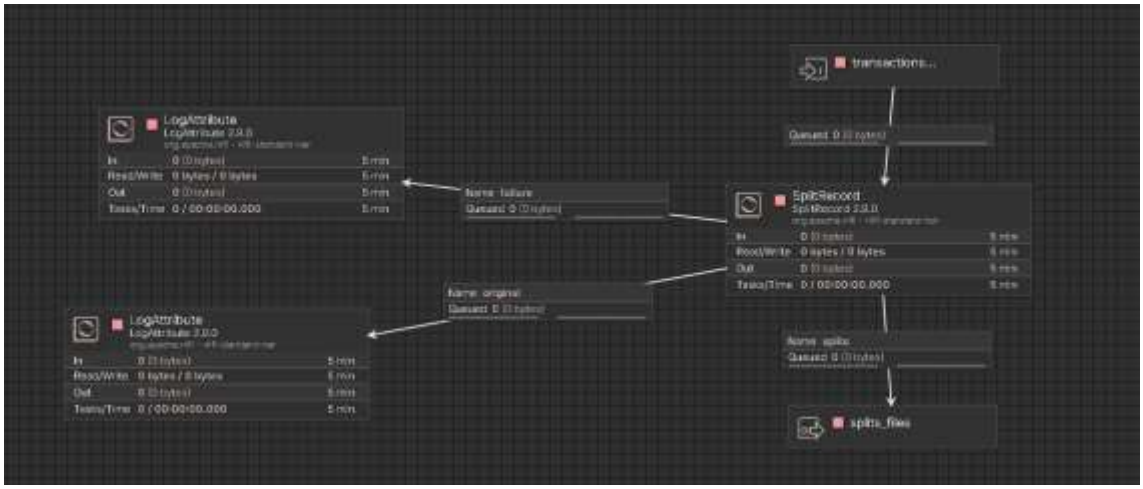
ListFile processor : I used it as the observer. It scans /lab_data/input every 5 seconds and when it sees a new file it passes it to FetchFile. I set it to use Entity Tracking so it remembers which files it already processed and does not pick up the same file twice.

FetchFile processor : I used it as the hunter. When ListFile tells it there is a file, it fetches it immediately. I set the Completion Strategy to Delete File after fetching so the directory stays clean. I also set it to retry on failure after penalized duration so temporary issues do not break the pipeline.

LogAttribute processor : Logs not found and permission denied files so I can trace issues without stopping the flow.

Process Group: file_chunking

Each incoming CSV file gets split into smaller chunks before transformation. I implemented this using SplitRecord with 1544 records per split — which keeps each chunk safely under the 64 KB limit.



SplitRecord processor : I chose SplitRecord because it understands the CSV schema — it does not blindly cut bytes, it splits on record boundaries so no row ever gets cut in half. It uses the CSVReader controller service to read and the CSVRecordSetWriter to write each chunk back as a proper CSV with headers.

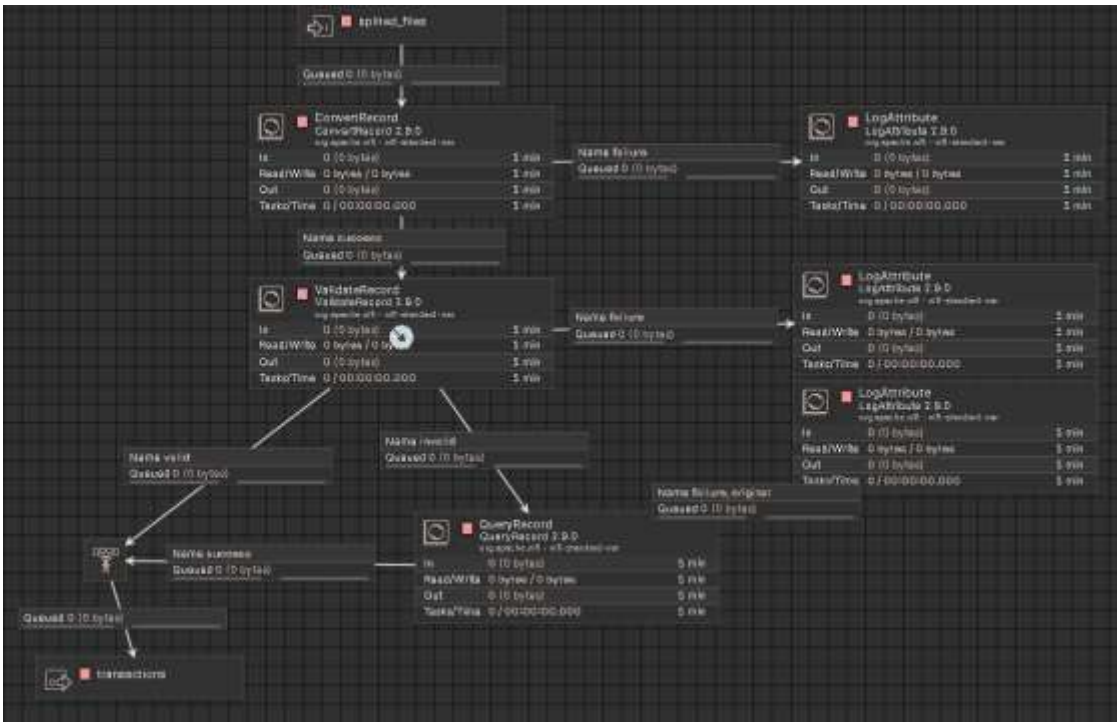
Ask me why 1544 records per split and not a fixed byte limit?

Because SplitRecord works at the record level, not the byte level. 1544 records gives approximately 60–63 KB per chunk for this schema, which stays comfortably under the 64 KB requirement. Using a record-based split also guarantees data integrity — no partial rows

.

Process Group: transformation

I built three layers of quality control here: Convert then Validate then Fix Nulls.



ConvertRecord : Reads the CSV using CSVReader and writes it as JSON using JsonRecordSetWriter. This is the main CSV-to-JSON conversion step. Both reader and writer share the same Avro schema so the field names and types are consistent.

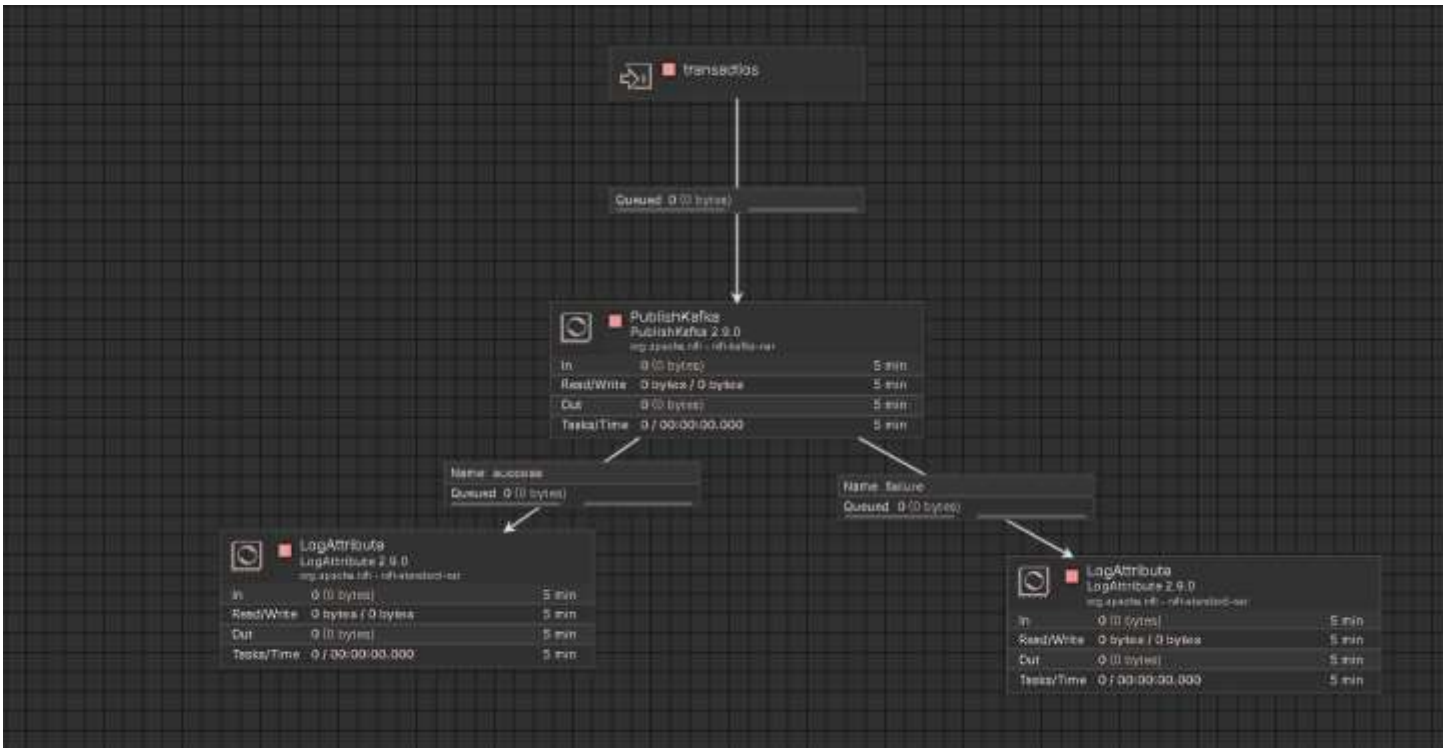
ValidateRecord : Takes the JSON output and validates every record against the BankTransaction Avro schema. Valid records go forward. Invalid records do not get thrown away — they get routed to QueryRecord for null-fixing instead.

QueryRecord : I used it to fix the invalid records. It runs a SQL query on the FlowFile to replace null values with safe defaults — amount becomes 0, currency becomes UNKNOWN, timestamp becomes 1970-01-01T00:00:00Z. After fixing, these records join the valid ones through a Funnel.

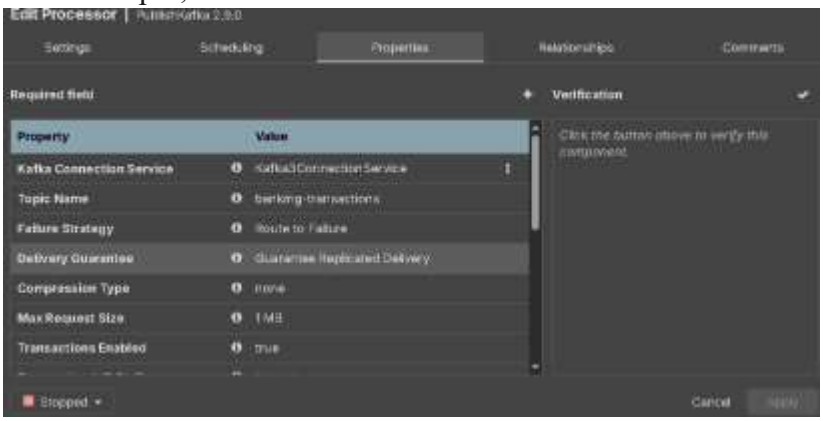
LogAttribute : Logs all failures, invalid records, and unmatched transactions so nothing gets silently dropped.

Kafka Integration

Process Group: to kafka



PublishKafka:I used it as producer that Reads the JSON files using JsonTreeReader and publishes each record to the banking-transactions topic,



create topic for the transactions:
I used 3 partitions for more effective storing and processing and I used the transaction_type field as the Kafka key to store all the transactions with the same transaction type in the same partion

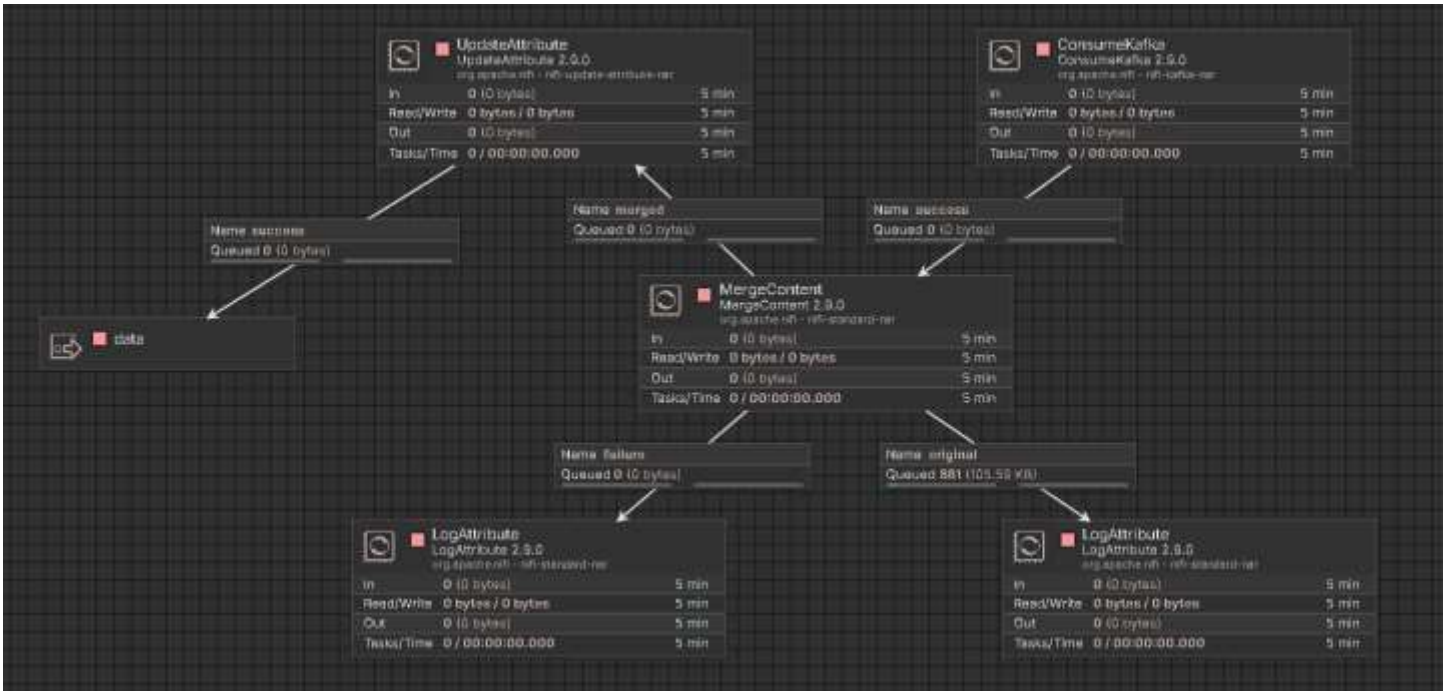
```
[appuser@kafka1 ~]$
[appuser@kafka1 ~]$ kafka-topics --create --topic banking-transactions \
> --bootstrap-server kafka1:19092 \
> --replication-factor 3 \
> --partitions 3
Created topic banking-transactions.
[appuser@kafka1 ~]$
```

Thr transactions in kafka UI :

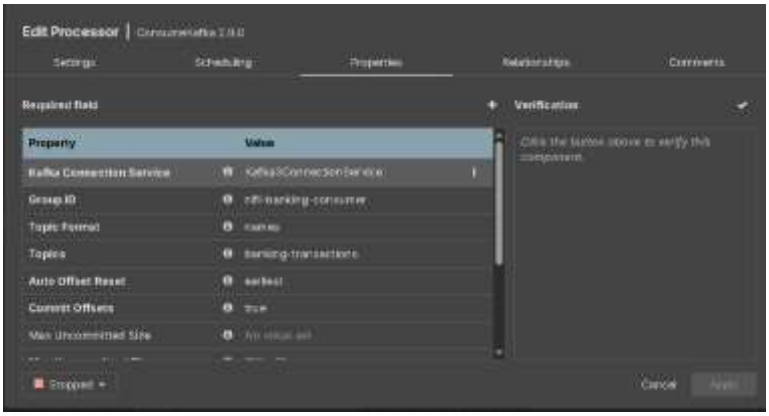
Offset	Partition	Timestamp	Key Preview	Value Preview
0	0	5/16/2026, 00:25:10		{"account_id":"ACC256","transaction_type":"WIT...
1	0	5/16/2026, 00:25:10		{"account_id":"ACC256","transaction_type":"WIT...
2	0	5/16/2026, 00:25:10		{"account_id":"ACC403","transaction_type":"INV...
3	0	5/16/2026, 00:25:10		{"account_id":"ACC031","transaction_type":"WIT...
4	0	5/16/2026, 00:25:10		{"account_id":"ACC256","transaction_type":"PAY...
5	0	5/16/2026, 00:25:10		{"account_id":"ACC031","transaction_type":"INV...
6	0	5/16/2026, 00:25:10		{"account_id":"ACC468","transaction_type":"WIT...
7	0	5/16/2026, 00:25:10		{"account_id":"ACC256","transaction_type":"INV...

Process Group: from kafka

This group consumes the messages back from Kafka, merges them into batches, and passes them to HDFS storage.



ConsumeKafka : Subscribes to the banking-transactions topic using Group ID: nifi-banking-consumer. I set auto.offset.reset to earliest so if NiFi restarts it picks up from where it left off. Max Uncommitted Time is 100 millis to minimize data loss on failure. isolation.level is set to read_committed so only fully committed Kafka transactions are consumed.



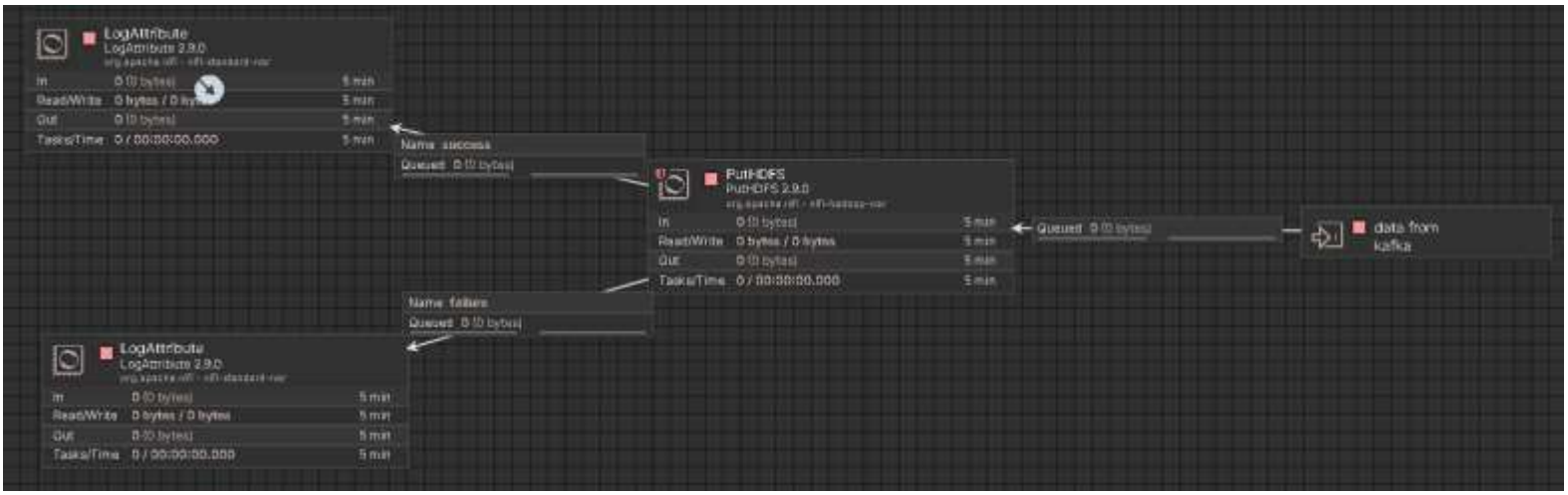
#MergeContent : I used it for merge Kafka messages into batches of up to 1000 records using Binary Concatenation. This reduces the number of small files written to HDFS and improves storage efficiency, because the processor face difficulty when it send the small files to HDFS

UpdateAttribute : Generates a unique filename for each batch using the pattern: transaction_${now():format('yyyyMMdd_HHmmss_SSS')}.json

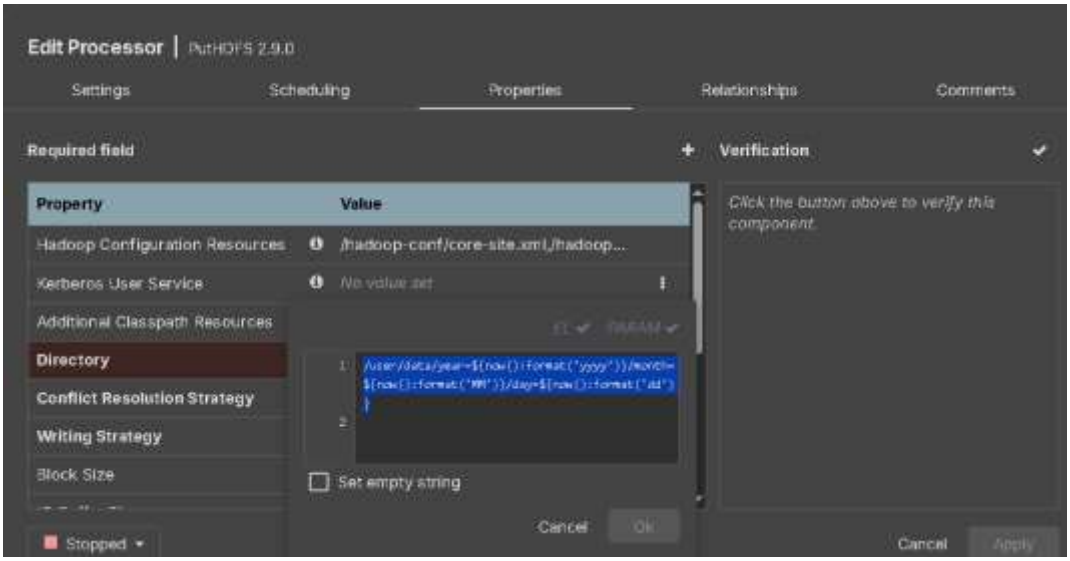
---

Process Group: to HDF

The final merged JSON batches get stored in HDFS with a date-partitioned directory structure. This makes it easy to query specific days and also works naturally with Hive or Spark partitioned reads.



PutHDFS: I used it to connects to Hadoop using the configuration files at /hadoop-conf/core-site.xml and /hadoop-conf/hdfs-site.xml,

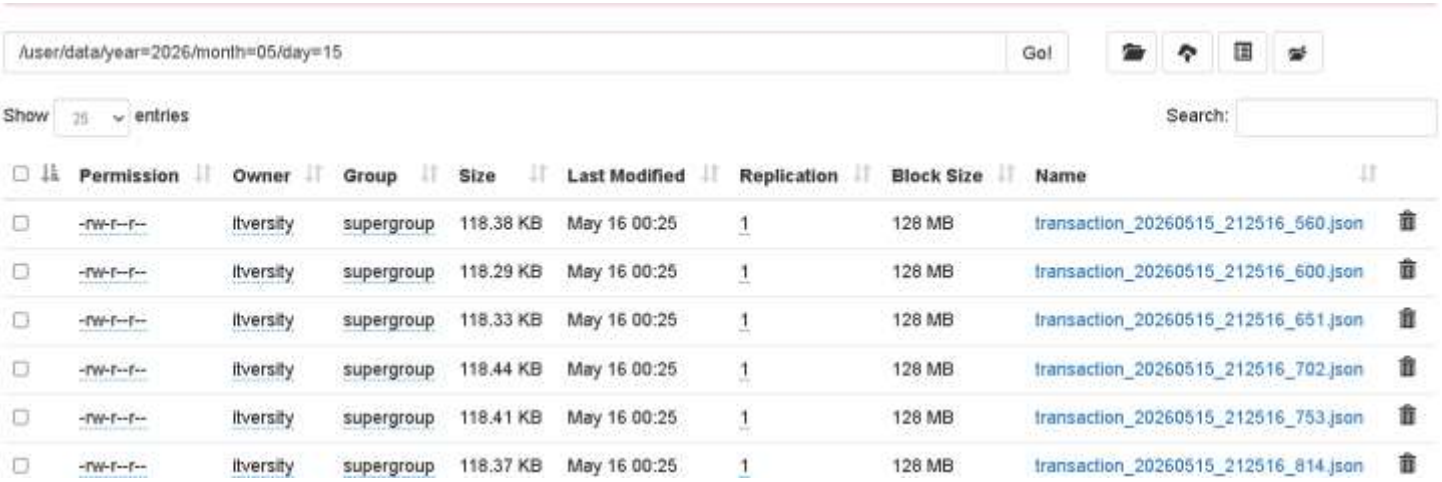


Hdfs directory structure:

```
/user/data/
  year=2026/
    month=05/
      day=11/
        transaction_20260511_143201_042.json
        transaction_20260511_143206_107.json
        transaction_20260511_143211_381.json
      day=12/
        ...
```

UpdateAttribute: Creates the filename attribute before PutHDFS runs — it generates the full timestamped filename that will be used as the HDFS file name.

LogAttribute: Logs success and failure for every file write so I have a complete audit trail of what landed in HDFS and what failed.



Monitoring and reliability:

Back Pressure: I set a limit of 3000 FlowFiles in some layers so data doesn't pile up during peak hours.

Penalty Duration: If a file fails, NiFi waits a little before retrying – this way we don't hammer the source systems.

Retry Handling: I configured most processors to retry up to 10 times.

Provenance Tracking: I can trace every FlowFile from the moment it enters until it leaves – great for debugging.

Error Handling: Every failure or invalid record is logged using LogAttribute so we can review them later.

Queue Management: I used prioritizers (like FIFO – First In First Out) to ensure older records are processed first.

Challenges Faced During Implementation

Schema Consistency Across Sources: the CSV files from the Python generator had slightly different field naming compared to what NiFi expected. I solved this by defining a single Avro schema in the CSVReader controller service and reusing it in both the CSVRecordSetWriter and JsonRecordSetWriter. This way no matter what comes in, it follows the same structure on the way out.

Null Values Breaking Validation: the generator intentionally creates records with null amounts and currencies. ValidateRecord with strict type checking would reject all of them. I solved this by routing invalid records to QueryRecord which uses COALESCE to substitute safe defaults, then merging them back with valid records through a Funnel. This way zero data is lost.

HDFS File Conflicts: If two NiFi threads try to write the same filename to HDFS simultaneously, the second one fails. I solved this using UpdateAttribute with a millisecond-precise timestamp in the filename pattern: transaction_${now():format('yyyyMMdd_HHmmss_SSS')}.json — the SSS millisecond component makes collisions practically impossible.

Chunking Without Breaking Records: a naive byte-level split would cut CSV rows in the middle. I used SplitRecord instead of SplitContent because SplitRecord understands the record structure and always splits on complete record boundaries. 1544 records per split was chosen to keep chunks under 64 KB for the average record size in this dataset